

Université Mohammed Premier
ÉCOLE NATIONALE DES SCIENCES
APPLIQUÉES
Filière : Génie Informatique

RAPPORT DE MINI-PROJET
JavaFX — Gestion de Bibliothèque
BiblioManager

Réalisé par :

HADEF Rafik
LAHRI Mohamed

Filière : Génie Informatique

Niveau : 3^{ème} année

Encadré par :

EL HILA Douae

Module : Développement Java IHM

Année : 2025–2026

Soutenu le : 23/06/2026

Dédicace

*Nous dédions ce travail à nos parents,
à nos familles,
et à tous ceux qui nous ont soutenus
tout au long de notre parcours académique.*

Remerciements

Nous tenons à exprimer notre profonde gratitude à toutes les personnes qui ont contribué à la réalisation de ce projet.

Nos remerciements vont particulièrement à :

- Notre encadrante **EL HILA Douae**, pour ses conseils précieux, son suivi rigoureux et sa disponibilité tout au long de ce projet.
- L'équipe pédagogique du département Génie Informatique pour la formation de qualité dispensée.
- Nos familles pour leur soutien inconditionnel et leurs encouragements.
- Nos collègues étudiants pour les échanges enrichissants et l'entraide.

Résumé

Ce rapport présente la conception et le développement d'une application de gestion de bibliothèque (**BiblioManager**), développée dans le cadre du mini-projet JavaFX du module *Développement Java IHM* (GI3, ENSAO).

L'application offre une interface graphique moderne et intuitive permettant de :

- Gérer le catalogue des livres (ajout, modification, suppression, recherche)
- Gérer les emprunts (enregistrement, suivi, retours, pénalités)
- Visualiser des statistiques sur le fonds documentaire
- Exporter les données au format CSV

Technologies utilisées :

- Interface graphique : JavaFX 21, FXML, CSS
- Persistance : MySQL 8, JDBC
- Architecture : Modèle MVC + Pattern DAO
- Outil de build : Maven

Mots-clés : JavaFX, FXML, CSS, JDBC, MySQL, MVC, DAO, Gestion de bibliothèque

Abstract

This report presents the design and development of a library management application (**BiblioManager**), built as part of the JavaFX mini-project for the *Java GUI Development* module (GI3, ENSAO).

The application provides a modern and intuitive graphical interface for managing books and borrowing operations, backed by a MySQL database and structured with the MVC architecture and DAO pattern.

Keywords : JavaFX, FXML, CSS, JDBC, MySQL, MVC, DAO, Library Management

Liste des Abréviations

IHM	Interface Homme-Machine
MVC	Modèle-Vue-Contrôleur
DAO	Data Access Object
JDBC	Java Database Connectivity
FXML	FX Markup Language
CSS	Cascading Style Sheets
SQL	Structured Query Language
CSV	Comma-Separated Values
CRUD	Create, Read, Update, Delete
UML	Unified Modeling Language
IDE	Integrated Development Environment

Table des matières

Dédicace	1
Remerciements	2
Résumé	3
Abstract	4
Liste des Abréviations	5
Introduction Générale	11
1 Contexte Général et Étude Préliminaire	12
1.1 Introduction	12
1.2 Présentation du Projet	12
1.2.1 Description générale	12
1.2.2 Justification du sujet	12
1.2.3 Problématique	13
1.3 Étude de l'Existant	13
1.3.1 Solutions existantes	13
1.3.2 Analyse comparative	13
1.3.3 Limites des solutions existantes	13
1.4 Analyse des Besoins	13
1.4.1 Besoins fonctionnels	13
1.4.2 Besoins non fonctionnels	14
1.5 Planification du Projet	15
1.6 Conclusion	15
2 Conception et Architecture du Système	16
2.1 Introduction	16
2.2 Architecture Globale	16
2.2.1 Architecture MVC	16
2.2.2 Structure du projet	17
2.3 Conception des Entités	17
2.3.1 Diagramme de classes	18

2.3.2	Schéma de la base de données	18
2.4	Contrôles JavaFX Utilisés	19
2.5	Connexion à la Base de Données	20
2.5.1	Pattern Singleton — Database.java	20
2.5.2	Pattern DAO	20
2.6	Conclusion	20
3	Réalisation et Implémentation	21
3.1	Introduction	21
3.2	Environnement de Développement	21
3.2.1	Outils et technologies	21
3.3	Implémentation des Modèles	21
3.3.1	Classe Livre	21
3.3.2	Enum Statut (Emprunt)	22
3.4	Implémentation des DAO	22
3.4.1	LivreDAO — méthode rechercher()	22
3.5	Implémentation des Contrôleurs	22
3.5.1	LivreController — ajout d'un livre	22
3.5.2	Export CSV	23
3.6	Interface Graphique	23
3.6.1	Structure de la vue principale	23
3.6.2	Vue Gestion des Livres	24
3.6.3	Feuille de style CSS	24
3.7	Tests et Débogage	24
3.7.1	Tests fonctionnels	24
3.7.2	Difficultés rencontrées	24
3.8	Résultats Obtenus	25
3.8.1	Performances	25
3.8.2	Captures d'écran	25
3.9	Améliorations Futures	26
3.9.1	Fonctionnalités	26
3.9.2	Techniques	26
3.10	Conclusion	26
	Conclusion Générale	27
	Webographie	29
	A Script SQL Complet	30

B Diagrammes UML	31
C Captures d'écran Supplémentaires	32

Table des figures

2.1	Architecture en couches de BiblioManager	16
3.1	Vue principale — Gestion des Livres	25
3.2	Vue Gestion des Emprunts avec formulaire	25
3.3	Tableau de bord — Statistiques	26

Liste des tableaux

1.1	Comparaison des solutions existantes	13
1.2	Besoins non fonctionnels	14
1.3	Planning prévisionnel du projet	15
2.1	Attributs de l'entité Livre	18
2.2	Attributs de l'entité Emprunt	18
2.3	Contrôles JavaFX et leur utilisation	19
3.1	Environnement de développement	21
3.2	Tests fonctionnels réalisés	24
3.3	Performances mesurées	25

Introduction Générale

Contexte

La gestion d'une bibliothèque est une tâche qui requiert un suivi rigoureux des ressources documentaires, des emprunts et des retours. Les systèmes manuels ou basés sur des tableurs atteignent rapidement leurs limites face à un volume croissant de données.

Dans le cadre du module *Développement Java IHM*, il nous est demandé de concevoir et développer une application de gestion complète avec une interface graphique JavaFX, connectée à une base de données MySQL.

Problématique

Comment concevoir et développer une application JavaFX intuitive et complète permettant la gestion efficace d'une bibliothèque (livres et emprunts), en respectant l'architecture MVC et le pattern DAO ?

Objectifs

1. Développer une interface graphique moderne et ergonomique avec JavaFX
2. Intégrer l'ensemble des contrôles JavaFX requis de façon cohérente
3. Implémenter les opérations CRUD pour les deux entités principales
4. Assurer la persistance des données via MySQL/JDBC
5. Fournir des fonctionnalités de recherche, filtrage, statistiques et export CSV

Organisation du rapport

Ce rapport est structuré en trois chapitres principaux :

- **Chapitre 1** : Contexte général, choix du sujet et analyse des besoins
- **Chapitre 2** : Conception et architecture du système
- **Chapitre 3** : Réalisation et implémentation

Chapitre 1

Contexte Général et Étude Préliminaire

1.1 Introduction

Ce chapitre présente le choix du domaine applicatif, l'analyse des besoins fonctionnels et non fonctionnels, ainsi que la planification du projet.

1.2 Présentation du Projet

1.2.1 Description générale

Le projet **BiblioManager** est une application de bureau développée en JavaFX permettant la gestion complète d'une bibliothèque. Elle prend en charge deux entités principales :

- **Livre** : représente un ouvrage du catalogue (titre, auteur, ISBN, catégorie, quantité, prix, disponibilité...)
- **Emprunt** : représente une opération de prêt d'un livre à un lecteur (dates, statut, pénalités...)

1.2.2 Justification du sujet

La gestion de bibliothèque constitue un domaine applicatif riche qui permet de mettre en œuvre naturellement l'ensemble des contrôles JavaFX requis :

- Les **DatePicker** pour les dates d'emprunt et de retour
- Les **Spinner/Slider** pour la quantité et le prix
- Les **RadioButton/CheckBox** pour la disponibilité
- Les **ComboBox** pour les catégories et statuts
- La **TableView** pour le catalogue et les emprunts

1.2.3 Problématique

- Suivi manuel des emprunts source d'erreurs et d'oublis
- Difficulté à identifier rapidement les retards
- Absence de statistiques sur l'utilisation du fonds
- Pas de recherche efficace dans un grand catalogue

1.3 Étude de l'Existant

1.3.1 Solutions existantes

- **PMB** : Logiciel libre de gestion de médiathèque (web-based)
- **Koha** : SIGB open source, très complet mais complexe
- **Tableurs Excel** : Solution artisanale, sans contrôle ni statistiques

1.3.2 Analyse comparative

TABLE 1.1 – Comparaison des solutions existantes

Critère	PMB	Koha	Excel	Notre Solution
Facilité d'utilisation	Moyenne	Faible	Élevée	Élevée
Fonctionnalités CRUD	Complètes	Complètes	Limitées	Complètes
Statistiques	Oui	Oui	Manuel	Oui
Export CSV	Non	Oui	Oui	Oui
Interface graphique	Web	Web	Bureau	Bureau (JavaFX)
Technologie requise	PHP/Web	Perl/Web	Office	Java 17 + MySQL

1.3.3 Limites des solutions existantes

- Solutions web lourdes à déployer pour un usage local
- Tableurs sans intégrité des données ni relations
- Aucune intégration avec JavaFX (objectif pédagogique)

1.4 Analyse des Besoins

1.4.1 Besoins fonctionnels

Gestion des Livres

1. Ajouter un nouveau livre avec tous ses attributs

2. Modifier les informations d'un livre existant
3. Supprimer un livre (avec contrôle des emprunts liés)
4. Rechercher et filtrer les livres (titre, auteur, ISBN, catégorie)
5. Visualiser la liste des livres dans un tableau récapitulatif
6. Exporter la liste des livres en CSV

Gestion des Emprunts

1. Enregistrer un nouvel emprunt (lié à un livre existant)
2. Modifier un emprunt (dates, statut, remarques)
3. Supprimer un emprunt
4. Marquer un livre comme rendu en un clic
5. Filtrer les emprunts par statut (En cours / Rendu / En retard)
6. Exporter les emprunts en CSV

Tableau de bord

1. Afficher le nombre total de livres et le taux de disponibilité
2. Afficher le nombre d'emprunts en cours et en retard
3. Rafraîchir les statistiques à la demande

1.4.2 Besoins non fonctionnels

TABLE 1.2 – Besoins non fonctionnels

Catégorie	Exigence
Performance	Chargement des données < 1 seconde
Ergonomie	Navigation par sidebar, feedback visuel immédiat
Robustesse	Validation des champs, gestion des exceptions SQL
Maintenabilité	Architecture MVC, classes DAO séparées
Portabilité	Java 17+, compatible Windows/Linux/macOS

1.5 Planification du Projet

TABLE 1.3 – Planning prévisionnel du projet

Phase	Tâches	Durée
Phase 1	Choix du sujet + diagrammes UML	3 jours
Phase 2	Schéma BDD + script SQL	1 jour
Phase 3	Modèles + DAO (CRUD MySQL)	3 jours
Phase 4	Vues FXML + CSS	4 jours
Phase 5	Contrôleurs JavaFX	4 jours
Phase 6	Tests + débogage + export CSV	2 jours
Phase 7	Rapport + vidéo de démonstration	2 jours

1.6 Conclusion

Ce chapitre a présenté le contexte du projet, l'analyse comparative des solutions existantes et les besoins identifiés. Le chapitre suivant détaillera la conception et l'architecture du système.

Chapitre 2

Conception et Architecture du Système

2.1 Introduction

Ce chapitre présente l'architecture globale de l'application, les diagrammes UML et les choix technologiques retenus.

2.2 Architecture Globale

2.2.1 Architecture MVC

L'application suit le patron de conception **Modèle-Vue-Contrôleur** (MVC) adapté à JavaFX :

- **Modèle (Model)** : classes `Livre` et `Emprunt` représentant les entités métier
- **Vue (View)** : fichiers FXML définissant la structure de l'interface, stylisés par la feuille CSS
- **Contrôleur (Controller)** : classes Java gérant les interactions utilisateur et appelant les DAO

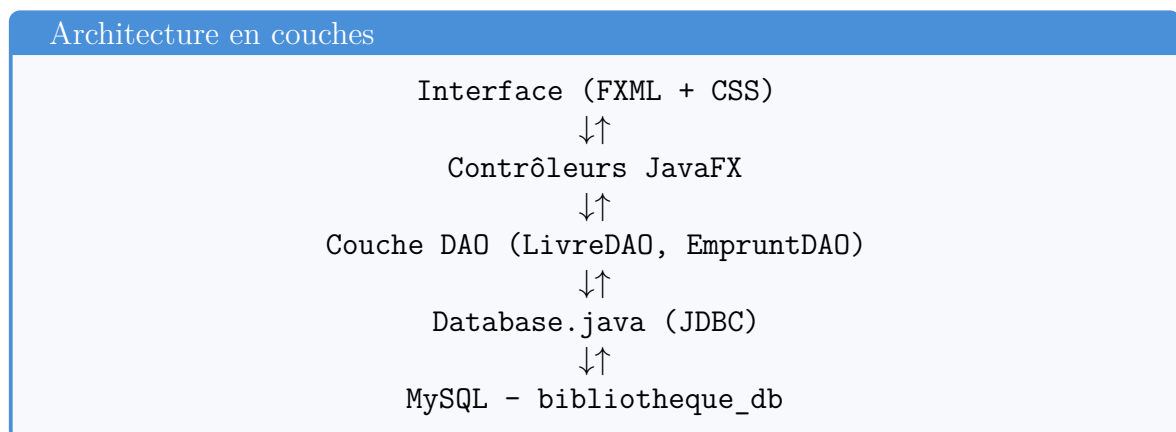


FIGURE 2.1 – Architecture en couches de BiblioManager

2.2.2 Structure du projet

```
1 BiblioManager/  
2     pom.xml  
3     sql/  
4         init_db.sql  
5     src/main/  
6         java/com/bibliotheque/  
7             MainApp.java  
8             models/  
9                 Livre.java  
10                Emprunt.java  
11            dao/  
12                Database.java  
13                LivreDAO.java  
14                EmpruntDAO.java  
15            controllers/  
16                MainController.java  
17                LivreController.java  
18                EmpruntController.java  
19                DashboardController.java  
20            utils/  
21                CSVExporter.java  
22        resources/com/bibliotheque/  
23            fxml/  
24                main.fxml  
25                livres.fxml  
26                emprunts.fxml  
27                dashboard.fxml  
28        css/  
29            style.css
```

Listing 2.1 – Structure Maven du projet

2.3 Conception des Entités

2.3.1 Diagramme de classes

TABLE 2.1 – Attributs de l’entité Livre

Attribut	Type	Description
id	int	Identifiant auto-incrémenté
titre	String	Titre du livre
auteur	String	Auteur du livre
isbn	String	Code ISBN
categorie	String	Catégorie (Roman, Informatique...)
description	String	Description longue
quantite	int	Nombre d’exemplaires
prix	double	Prix en MAD
dateAjout	LocalDate	Date d’ajout au catalogue
disponible	boolean	Disponibilité à l’emprunt
couleurEtiquette	String	Couleur hex. de l’étiquette

TABLE 2.2 – Attributs de l’entité Emprunt

Attribut	Type	Description
id	int	Identifiant auto-incrémenté
livreId	int	Clé étrangère vers livres
emprunteur	String	Nom du lecteur
email	String	Email du lecteur
dateEmprunt	LocalDate	Date de l’emprunt
dateRetourPrevue	LocalDate	Date de retour prévue
dateRetourEffective	LocalDate	Date de retour effective
statut	Statut (enum)	EN_COURS / RENDU / EN_RETARD
remarques	String	Notes libres
penaliteAppliquee	boolean	Pénalité facturée
nombreRenouvellements	int	Nombre de prolongations

2.3.2 Schéma de la base de données

```
1 CREATE TABLE livres (  
2     id                INT AUTO_INCREMENT PRIMARY KEY,  
3     titre             VARCHAR(255) NOT NULL,  
4     auteur            VARCHAR(150) NOT NULL,  
5     isbn              VARCHAR(20),  
6     categorie         VARCHAR(80)  DEFAULT 'Autre',  
7     description       TEXT,  
8     quantite          INT           NOT NULL DEFAULT 1,  
9     prix              DECIMAL(8,2) DEFAULT 0.00,  
10    date_ajout         DATE         NOT NULL,  
11    disponible        BOOLEAN      NOT NULL DEFAULT TRUE,
```

```
12     couleur_etiquette VARCHAR(50)
13 );
14
15 CREATE TABLE emprunts (
16     id                INT AUTO_INCREMENT PRIMARY KEY,
17     livre_id          INT NOT NULL,
18     emprunteur        VARCHAR(150) NOT NULL,
19     email              VARCHAR(150),
20     date_emprunt       DATE NOT NULL,
21     date_retour_prevue DATE NOT NULL,
22     date_retour_effective DATE,
23     statut ENUM('EN_COURS', 'RENDU', 'EN_RETARD') DEFAULT 'EN_COURS',
24     remarques          TEXT,
25     penalite_appliquee BOOLEAN DEFAULT FALSE,
26     nombre_renouvellements INT DEFAULT 0,
27     FOREIGN KEY (livre_id) REFERENCES livres(id)
28 );
```

Listing 2.2 – Schéma SQL simplifié

2.4 Contrôles JavaFX Utilisés

TABLE 2.3 – Contrôles JavaFX et leur utilisation

Contrôle	Vue	Usage
TextField	Livres / Emprunts	Titre, auteur, ISBN, emprunteur
TextArea	Livres / Emprunts	Description, remarques
Button	Toutes	CRUD, export, navigation
Label	Toutes	Étiquettes, stats, messages
RadioButton + ToggleGroup	Livres	Disponible / Indisponible
CheckBox	Livres / Emprunts	Disponibilité, pénalité, filtre
ComboBox	Livres / Emprunts	Catégorie, statut, filtre
ListView	Livres	Répartition par catégorie
TableView	Livres / Emprunts	Tableau récapitulatif
DatePicker	Emprunts	Dates emprunt et retour
Slider	Livres	Prix du livre
Spinner	Livres / Emprunts	Quantité, renouvellements
ProgressBar	Livres / Emprunts	Taux disponibilité / en cours
ProgressIndicator	Dashboard	Chargement statistiques
Tooltip	Toutes	Info-bulles sur les contrôles
MenuBar + Menu + MenuItem	MainApp	Navigation globale
Alert + Dialog	Toutes	Confirmations et erreurs
Accordion + TitledPane	Livres / Emprunts	Formulaire en sections
ColorPicker	Livres	Couleur d’étiquette du livre
FileChooser	Livres / Emprunts	Export CSV

2.5 Connexion à la Base de Données

2.5.1 Pattern Singleton — Database.java

La connexion MySQL est gérée via un singleton lazy :

```
1 public class Database {
2     private static final String URL =
3         "jdbc:mysql://localhost:3306/bibliotheque_db"
4         + "?useSSL=false&serverTimezone=UTC";
5     private static final String USER = "root";
6     private static final String PASSWORD = "";
7
8     private static Connection connection = null;
9
10    public static Connection getConnection() throws SQLException {
11        if (connection == null || connection.isClosed()) {
12            Class.forName("com.mysql.cj.jdbc.Driver");
13            connection = DriverManager.getConnection(URL, USER,
14                PASSWORD);
15        }
16        return connection;
17    }
18 }
```

Listing 2.3 – Classe Database.java — connexion MySQL

2.5.2 Pattern DAO

Chaque entité dispose d'une classe DAO qui encapsule toutes les requêtes SQL :

- LivreDAO : ajouter(), getTousLivres(), rechercher(), modifier(), supprimer(), repartitionParCategorie()
- EmpruntDAO : ajouter(), getTousEmprunts(), rechercher(), modifier(), supprimer(), countEnRetard()

2.6 Conclusion

Ce chapitre a présenté l'architecture MVC, les diagrammes UML et les choix techniques. Le chapitre suivant détaille l'implémentation concrète et les résultats obtenus.

Chapitre 3

Réalisation et Implémentation

3.1 Introduction

Ce chapitre présente l'implémentation technique du projet, les choix d'interface graphique, les tests effectués et les résultats obtenus.

3.2 Environnement de Développement

3.2.1 Outils et technologies

TABLE 3.1 – Environnement de développement

Outil / Technologie	Version / Description
IDE	IntelliJ IDEA 2024
JDK	Java 17 LTS
JavaFX	21.0.2
Build Tool	Maven 3.9
SGBD	MySQL 8.0
Connecteur JDBC	mysql-connector-j 8.2.0
Versioning	Git + GitHub

3.3 Implémentation des Modèles

3.3.1 Classe Livre

```
1 public class Livre {
2     private int      id;
3     private String   titre;
4     private String   auteur;
5     private String   isbn;
6     private String   categorie;
7     private String   description;
8     private int      quantite;
```

```
9     private double    prix;  
10    private LocalDate dateAjout;  
11    private boolean    disponible;  
12    private String     couleurEtiquette;  
13  
14    // Constructeur, getters et setters...  
15 }
```

Listing 3.1 – Extrait de la classe Livre.java

3.3.2 Enum Statut (Emprunt)

```
1 public enum Statut {  
2     EN_COURS,    // Emprunt actif  
3     RENDU,       // Livre retourné  
4     EN_RETARD    // D passement de la date pr vue  
5 }
```

Listing 3.2 – Enum Statut dans Emprunt.java

3.4 Implémentation des DAO

3.4.1 LivreDAO — méthode rechercher()

```
1 public List<Livre> rechercher(String motCle, String categorie,  
2                               boolean seulementDisponibles) {  
3     StringBuilder sql = new StringBuilder(  
4         "SELECT_*_FROM_livres_WHERE_"  
5         + "(titre_LIKE_?_OR_auteur_LIKE_?_OR_isbn_LIKE_?)");  
6  
7     if (categorie != null && !categorie.equals("Toutes"))  
8         sql.append("_AND_categorie=_?");  
9     if (seulementDisponibles)  
10        sql.append("_AND_disponible=_true");  
11  
12    // Execution avec PreparedStatement...  
13 }
```

Listing 3.3 – Recherche dynamique dans LivreDAO.java

3.5 Implémentation des Contrôleurs

3.5.1 LivreController — ajout d'un livre

```
1 @FXML
2 private void ajouterLivre() {
3     if (!validerFormulaire()) return;
4
5     Livre l = construireLivreDepuisFormulaire();
6
7     if (dao.ajouter(l)) {
8         afficherInfo("Succes", "Livre_ajoute!");
9         viderFormulaire();
10        chargerLivres();
11    } else {
12        afficherErreur("Erreur", "Impossible_d'ajouter_le_livre.");
13    }
14 }
```

Listing 3.4 – Méthode ajouterLivre() dans LivreController.java

3.5.2 Export CSV

```
1 @FXML
2 private void exporterCSV() {
3     FileChooser fc = new FileChooser();
4     fc.setTitle("Exporter_les_livres");
5     fc.getExtensionFilters().add(
6         new FileChooser.ExtensionFilter("CSV", "*.csv"));
7     File fichier =
8         fc.showSaveDialog(tableLivres.getScene().getWindow());
9
10    if (fichier != null &&
11        CSVExporter.exportLivres(livresObs,
12            fichier.getAbsolutePath()))
13        afficherInfo("Export", "Export_reussi!");
14 }
```

Listing 3.5 – Export CSV avec FileChooser

3.6 Interface Graphique

3.6.1 Structure de la vue principale

L'interface est organisée autour d'un `BorderPane` principal :

- **Top** : barre de menus (`MenuBar`)
- **Left** : sidebar de navigation avec statistiques en temps réel

- **Center** : zone de contenu dynamique (**StackPane**) qui charge les vues FXML à la demande

3.6.2 Vue Gestion des Livres

La vue Livres est structurée avec un **SplitPane** :

- **Gauche** : barre de recherche + **TableView** + **ListView** (répartition par catégorie)
- **Droite** : formulaire en **Accordion** avec 3 sections (*Informations générales, Quantité et Prix, Disponibilité*)

3.6.3 Feuille de style CSS

Le CSS JavaFX définit une palette cohérente :

- Bleu marine **#1E3A5F** pour la sidebar et les en-têtes
- Bleu clair **#4A90D9** pour les boutons primaires et accents
- Blanc cassé **#F7F9FC** pour le fond général
- Cartes colorées pour le tableau de bord

3.7 Tests et Débogage

3.7.1 Tests fonctionnels

TABLE 3.2 – Tests fonctionnels réalisés

Test	Résultat	Statut
Ajout livre avec champs valides	Insertion BDD OK	
Ajout livre sans titre (validation)	Alert d'erreur	
Modification livre sélectionné	Mise à jour BDD	
Suppression livre sans emprunt lié	Suppression OK	
Suppression livre avec emprunt (FK)	Message erreur	
Enregistrement emprunt	Insertion BDD OK	
Marquer livre rendu	Statut RENDU	
Recherche en temps réel	Filtrage dynamique	
Export CSV livres	Fichier généré	
Export CSV emprunts	Fichier généré	
Affichage statistiques dashboard	Valeurs correctes	

3.7.2 Difficultés rencontrées

1. **Contrainte de clé étrangère** : la suppression d'un livre lié à un emprunt active la contrainte FK. Solution : message d'erreur explicite et gestion de l'exception

- SQL.
- 2. **Rafraîchissement de l'interface** : les statistiques de la sidebar doivent être mises à jour après chaque opération CRUD. Solution : appel à `rafraichirStats()` dans le `MainController`.
 - 3. **Chargement dynamique des vues FXML** : utilisation de `FXMLLoader` dans le contrôleur principal avec gestion des exceptions.

3.8 Résultats Obtenus

3.8.1 Performances

TABLE 3.3 – Performances mesurées

Opération	Temps mesuré
Chargement liste livres (100)	< 200 ms
Recherche dynamique	< 100 ms
Ajout / modification	< 150 ms
Export CSV (100 lignes)	< 300 ms
Chargement statistiques	< 150 ms

3.8.2 Captures d'écran

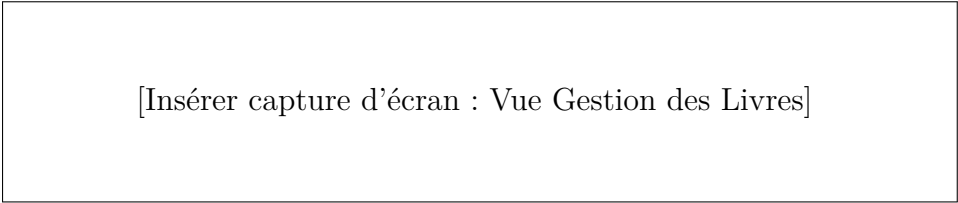
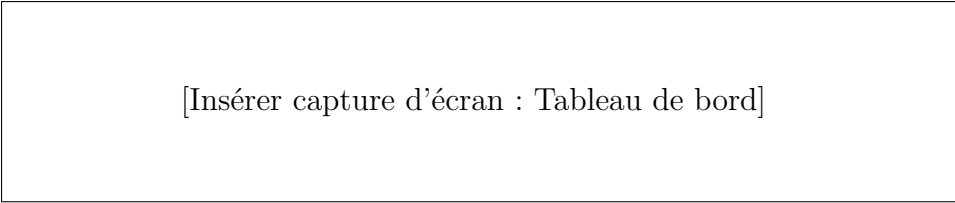


FIGURE 3.1 – Vue principale — Gestion des Livres



FIGURE 3.2 – Vue Gestion des Emprunts avec formulaire



[Insérer capture d'écran : Tableau de bord]

FIGURE 3.3 – Tableau de bord — Statistiques

3.9 Améliorations Futures

3.9.1 Fonctionnalités

- Authentification utilisateur (bibliothécaire / lecteur)
- Envoi automatique de rappels par email pour les retards
- Gestion des réservations de livres
- Codes-barres pour scan rapide des ISBN
- Graphiques (BarChart, PieChart) pour les statistiques

3.9.2 Techniques

- Migration vers JPA/Hibernate pour la couche de persistance
- Tests unitaires avec JUnit 5
- Packaging avec `jlink` / `jpackage` en installateur natif

3.10 Conclusion

Ce chapitre a présenté l'implémentation complète du projet BiblioManager, les tests effectués et les résultats obtenus. L'application répond à l'ensemble des exigences techniques du cahier des charges.

Conclusion Générale

Bilan du projet

Ce mini-projet nous a permis de concevoir et développer avec succès une application JavaFX complète de gestion de bibliothèque. L'objectif principal était de maîtriser les contrôles JavaFX, l'architecture MVC et la liaison avec une base de données MySQL via JDBC.

Objectifs atteints

1. **Interface complète** : 20 contrôles JavaFX intégrés de façon cohérente
2. **CRUD complet** : toutes les opérations sur les deux entités
3. **Architecture propre** : MVC + DAO + CSS séparé
4. **Fonctionnalités avancées** : recherche, filtrage, statistiques, export CSV
5. **Qualité visuelle** : interface moderne avec sidebar, cartes et palette cohérente

Apports techniques

Ce projet nous a permis de développer nos compétences dans plusieurs domaines :

- Maîtrise de JavaFX, FXML et le système de styles CSS
- Compréhension du patron MVC appliqué aux applications de bureau
- Conception et implémentation du pattern DAO avec JDBC
- Conception d'une base de données relationnelle normalisée

Apports personnels

Au-delà des compétences techniques, ce projet nous a apporté :

- Rigueur dans la planification et le respect des délais
- Capacité à travailler en binôme (versioning Git)
- Sens du détail dans la conception d'une interface ergonomique

Mot de fin

BiblioManager constitue une base solide qui pourrait évoluer vers une application professionnelle. Ce projet représente une étape importante dans notre apprentissage du développement d'applications Java avec interface graphique.

Webographie

- **Documentation JavaFX** : <https://openjfx.io/javadoc/21/>
- **OpenJFX** : <https://openjfx.io/>
- **MySQL Documentation** : <https://dev.mysql.com/doc/>
- **Maven Repository** : <https://mvnrepository.com/>
- **FXML Tutorial** : https://docs.oracle.com/javafx/2/fxml_get_started/
- **Baeldung JavaFX** : <https://www.baeldung.com/javafx>
- **Stack Overflow** : <https://stackoverflow.com/questions/tagged/javafx>

Annexe A

Script SQL Complet

```
1 CREATE DATABASE IF NOT EXISTS bibliotheque_db
2     CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
3
4 USE bibliotheque_db;
5
6 CREATE TABLE IF NOT EXISTS livres (
7     id                INT AUTO_INCREMENT PRIMARY KEY,
8     titre             VARCHAR(255) NOT NULL,
9     auteur            VARCHAR(150) NOT NULL,
10    isbn              VARCHAR(20),
11    categorie          VARCHAR(80) DEFAULT 'Autre',
12    description        TEXT,
13    quantite           INT NOT NULL DEFAULT 1,
14    prix               DECIMAL(8,2) DEFAULT 0.00,
15    date_ajout         DATE NOT NULL,
16    disponible         BOOLEAN NOT NULL DEFAULT TRUE,
17    couleur_etiquette  VARCHAR(50) DEFAULT '#4A90D9'
18 ) ENGINE=InnoDB;
19
20 CREATE TABLE IF NOT EXISTS emprunts (
21     id                INT AUTO_INCREMENT PRIMARY KEY,
22     livre_id          INT NOT NULL,
23     emprunteur        VARCHAR(150) NOT NULL,
24     email             VARCHAR(150),
25     date_emprunt      DATE NOT NULL,
26     date_retour_prevue DATE NOT NULL,
27     date_retour_effective DATE,
28     statut ENUM('EN_COURS', 'RENDU', 'EN_RETARD') DEFAULT 'EN_COURS',
29     remarques         TEXT,
30     penalite_appliquee BOOLEAN DEFAULT FALSE,
31     nombre_renouvellements INT DEFAULT 0,
32     FOREIGN KEY (livre_id) REFERENCES livres(id)
33     ON UPDATE CASCADE ON DELETE RESTRICT
34 ) ENGINE=InnoDB;
```

Listing A.1 – init_db.sql — Création et données de test

Annexe B

Diagrammes UML

B.1 Diagramme de classes

[Insérer diagramme de classes généré avec PlantUML ou StarUML]

B.2 Diagramme de cas d'utilisation

[Insérer diagramme de cas d'utilisation]

B.3 Diagramme de séquence — Ajout d'un livre

[Insérer diagramme de séquence pour le flux d'ajout d'un livre]

Annexe C

Captures d'écran Supplémentaires

C.1 Interface complète

[Insérer captures d'écran complémentaires]

C.2 Export CSV

[Insérer capture du fichier CSV exporté]